IoT SmartSort Architecture

AI-Powered Smart Waste Management System with IoT Integration for Bope-Poddala Divisional Secretariat
University of Ruhuna | Bachelor of Computer Science - Department of Computer Science
Design Baseline - Version 1.0 - October 2026

1. Purpose and Prototype Boundary

SmartSort is a controlled IoT prototype/public-station concept. It accepts **one waste item at a time**, captures an image, classifies the item into **Plastic, Paper, Metal, or Organic**, directs it to the corresponding compartment, monitors fill levels, and reports device telemetry to EcoCollect.

Boundary: the prototype uses fixed camera positioning and controlled lighting. It does not claim reliable sorting of mixed, dirty, overlapping, or unsupported waste.

2. Overall SmartSort Architecture

PHYSICAL SMARTSORT STATION		
Input Chamber **+ controlled lighting** **+ item sensor**	**ESP32-CAM** **image capture**	**ESP32 Controller** **servo + sensors + MQTT**
Waste item **single-item input**	**JPEG image** **over Wi-Fi/HTTP**	**Servo sorting flap** **4 ultrasonic sensors**
Plastic | Paper | Metal | Organic **four compartments**	**FastAPI AI Service** **OpenCV + YOLO** **classification + confidence**	**MQTT / Mosquitto** **status, fill level,** **sort event, error**
PostgreSQL **devices, events, readings, alerts**	**Express.js + TypeScript** **business API and integration**	**Admin Dashboard** **monitoring, alerts, history**

Primary data paths: ESP32-CAM sends the captured image to the FastAPI vision endpoint. The classification result is returned to the device/controller for actuation. Device telemetry and fill levels are published through MQTT. Express.js consumes/manages application data and stores operational records in PostgreSQL for the Admin Dashboard.

3. End-to-End Sorting Workflow

1	**Item insertion** User inserts one supported waste item into the controlled input chamber.
2	**Detection** IR/proximity sensor detects the item and initiates the capture sequence.
3	**Image capture** ESP32-CAM captures a JPEG image under controlled lighting.
4	**AI request** Image is sent to the FastAPI SmartSort vision endpoint.
5	**Classification** OpenCV/YOLO pipeline predicts Plastic, Paper, Metal, or Organic and returns a confidence score.
6	**Decision** If confidence is above the configured threshold, the category is accepted; otherwise the item is treated as unknown/rejected.
7	**Mechanical sorting** ESP32 controller moves the servo/flap to the correct compartment.
8	**Reset** After the item falls, the mechanism returns to its neutral position.
9	**Telemetry** Sorting event and device status are published through MQTT.
10	**Fill monitoring** Ultrasonic sensors measure the four compartment fill levels.
11	**Persistence** Backend stores device, event, reading and alert data in PostgreSQL.
12	**Monitoring** Admin Dashboard shows station status, fill levels, latest classification and alerts.

4. Hardware Architecture

Component	Responsibility
ESP32-CAM + OV2640	Capture waste images and communicate with the classification service.
ESP32 Dev Board	Main controller for item sensor, servo mechanism, ultrasonic sensors, device state and MQTT.
Servo motor / sorting flap	Direct the accepted item into one of four compartments.
4 x ultrasonic sensors	Estimate fill level for Plastic, Paper, Metal and Organic compartments.
IR / proximity sensor	Detect item insertion and trigger the sorting sequence.
LED lighting	Provide consistent illumination for image classification.
Power supply	Provide stable, isolated power suitable for ESP32 boards, sensors and servo load.
Chute + four compartments	Provide controlled physical path and waste separation.

Design choice: using a separate ESP32 controller alongside the ESP32-CAM simplifies pin availability, servo/sensor control, debugging and fault isolation.

5. AI Classification Architecture

The main computer-vision model runs on the server/laptop rather than directly on the ESP32-CAM. This keeps the embedded device simple and allows the team to train, evaluate and replace the model without rewriting the sorting firmware.

```
ESP32-CAM
| JPEG image
v
FastAPI: POST /api/v1/smartsort/classify
|
OpenCV preprocessing -> YOLO / classifier
|
+--> category: plastic
+--> confidence: 0.94
|
v
ESP32 Controller -> servo position -> compartment
```

Confidence rule: the architecture should define a configurable threshold. Low-confidence predictions should not be presented as reliable classifications; use an unknown/reject/manual fallback path.

6. MQTT Communication Design

Topic	Purpose
ecocollect/smartsort/SS001/status	Online/offline, firmware/device state and last-seen information.
ecocollect/smartsort/SS001/fill-level	Four compartment fill percentages.
ecocollect/smartsort/SS001/sort-event	Accepted classification, confidence and event timestamp.
ecocollect/smartsort/SS001/error	Sensor, servo, camera, network or device errors.

Example fill-level payload:

```
{
"deviceId": "SS001",
"plastic": 72,
"paper": 41,
"metal": 25,
"organic": 83,
"timestamp": "2026-10-07T10:15:00"
}
```

7. Backend and Database Interfaces

Interface	Responsibility
FastAPI /api/v1/smartsort/classify	Receive image and return category + confidence.
Express GET /api/smartsort/devices	List SmartSort stations/devices.
Express GET /api/smartsort/:id/status	Return current device and connectivity status.
Express GET /api/smartsort/:id/events	Return sorting-event history.
Express GET /api/smartsort/:id/fill-levels	Return latest/historical fill-level readings.
Express GET /api/smartsort/alerts	Return active SmartSort alerts.

8. Logical Data Model

Entity	Key information
SmartSortDevice	id, device_code, location, status, last_seen
SmartSortCompartment	id, device_id, waste_type, capacity, current_fill_level
SortingEvent	id, device_id, predicted_category, confidence, timestamp
FillLevelReading	id, compartment_id, fill_percentage, timestamp
DeviceAlert	id, device_id, alert_type, message, status, timestamp

9. Admin Dashboard View

The SmartSort Monitoring page should expose operational information rather than raw sensor values only. A typical station card should include station/location, online/offline state, fill level per compartment, warning thresholds, last classification and confidence, recent sorting counts, last-seen time, and active errors.

SS001 - Example Public Station **ONLINE**	**Fill levels** Plastic 72% Paper 41% Metal 25% Organic 83% - COLLECTION ALERT	**Last sort** Plastic Confidence: 94% **Today** Plastic 37 | Paper 22 Metal 11 | Organic 31

10. Design Constraints and Assumptions

• One item is processed at a time.

• Initial supported classes: Plastic, Paper, Metal, Organic.

• Fixed camera and controlled lighting are used to improve repeatability.

• Server/laptop connectivity is required for normal AI inference; loss-of-network behavior must be defined during implementation.

• Fill percentage from ultrasonic sensors is an estimate and requires calibration for the physical compartments.

• Servo movement must be mechanically constrained and tested to prevent jams.

• The prototype is a research/public-station demonstration and is not represented as an industrial mixed-waste sorter.

11. Sprint 1 Design Deliverable Checklist

■ Overall SmartSort architecture diagram

■ Hardware component list and responsibilities

■ End-to-end sorting sequence

■ AI classification interface and confidence handling

■ MQTT topic/payload design

■ Backend/API integration boundaries

■ Logical database entities

■ Dashboard monitoring requirements

■ Prototype constraints and assumptions

Design status: This document is the initial architecture baseline for the Jira task “Design IoT SmartSort Architecture”. Implementation details may be refined after component testing, but major scope changes should be reviewed against

the project WBS, schedule, and December 31 technical-completion target.